

Exercise: Biomedical Hybrid Search

Build and evaluate a RAG application over rag-mini-bioasq

Goal

Build a small biomedical search and question-answering application using rag-datasets/rag-mini-bioasq.

Your application should retrieve useful passages, show the evidence to the user, and use an LLM to write a grounded final answer with citations.

Dataset

Use the questions, answers, relevant passage IDs, and passage corpus.

Product

A search UI for biomedical questions with a clear evidence view.

Decision

Use offline results to choose the best retrieval configuration.

Step 1: build retrieval (20 marks)

Implement three retrieval modes over the same indexed passages:

1. Lexical search, such as BM25.
2. Dense vector search.
3. Hybrid search that combines both signals.

Add **query expansion** before retrieval. Each user query should generate a small set of useful alternate queries, while retaining the original query.

Record the retrieved passage IDs and scores for every search.

class: balanced bioasq task

Step 2: build the user interface (20 marks)

Your UI needs:

- A biomedical-question input and search action.
- The original query and its expansions.
- Ranked evidence passages with IDs, scores, and source links where available.
- A final LLM answer with inline citations to retrieved passage IDs.
- An explicit “insufficient evidence” state.

The final LLM call must answer only from the selected retrieved passages. You may host the UI locally or deploy it to a server such as Netlify.

Step 3: evaluate offline (20 marks)

EXERCISE TASK

Create a fixed set of **100 queries** from the dataset. Include a mix of yes/no, definition, list, treatment, mechanism, and effect questions where possible.

Compare at least:

Configuration	Required
Lexical	Yes
Dense	Yes
Hybrid	Yes
Hybrid + query expansion	Yes
Your strongest variant	Yes

Use the same 100 query IDs for every run.

Step 4: score retrieval and answers (20 marks)

Use both evaluation layers.

Traditional retrieval metrics

Measure Recall@5, Recall@10, MRR@10, nDCG@10, latency, and cost using `relevant_passage_ids`.

LLM-judge evaluation

Use RAGAS, DeepEval, or an equivalent framework to assess answer correctness, groundedness, and context relevance.

Manual review

Inspect examples where the judge and retrieval metrics disagree. Check citation validity in code.

Keep the judge prompt, model, and settings fixed across runs.

Step 5: system design (20 marks)

Create a system-design diagram for **your** solution and include it in the submission.

Your diagram should show:

- The offline data path: dataset loading, passage preparation, and the indexes you build.
- The online query path: query expansion, lexical and dense retrieval, hybrid fusion, selected context, and the final LLM call.
- The user-facing path: UI input, evidence display, final answer, and citations.
- The experiment path: how you log configurations and reuse the same pipeline for the 100-query evaluation.

Briefly explain the main choices in your README: storage or vector database, embedding model, fusion method, final-answer model, and where you enforce citation validation.

Submission

Submit:

1. A working UI and source code.
2. The fixed 100-query evaluation set and reproducible evaluation script.
3. A short report with one comparison table.
4. Your selected **best hybrid configuration**, backed by the experiment results.

Your report should state what won, why it won, and the trade-off in latency or cost. Include a few successful searches and a few failure cases.

Dataset: [rag-datasets/rag-mini-bioasq](#) on Hugging Face